Martin Ek, Andreas Lund, Prodip Kumar Das

October 12, 2026

Maintenance Programming
Progress Report 2 - Group 4

Contents

1 Progress so far

1.1 Renewing the HTTPS test certificates

1.2 Verifying the fix on macOS

1.3 Running the complete test suite

1.4 Further fixes

1.5 Current results

2 Outlook

2.1 Adjustments to the plan

2.2 Planned maintenance types

2.3 Next steps

3 Use of generative AI tools

Martin Ek, Andreas Lund, Prodip Kumar Das

October 12, 2026

1 Progress so far

1.1 Renewing the HTTPS test certificates

In Report 1, we found that the strict HTTPS tests failed because the test CA certificate, tests/ssl/ca/ca.crt, expired in February 2022. We renewed the CA certificate and generated a new server certificate and key signed by it. Strict certificate validation was kept, and request.js was not changed.

On Windows, tests/test-https.js then passed all 20 assertions, compared with 10 of 20 before.

```
1..20
# tests 20
# pass  20
# ok
```

Listing 1: tests/test-https.js after the renewal (Windows).

1.2 Verifying the fix on macOS

In order to check if the fix was just dependent on one machine, the tests were repeated on macOS with Node.js 22.21.1 and OpenSSL 3.6.2. The renewed certificate chain verified correctly, but the test server crashed before any test actually ran.

```
Error: error:0A00018F:SSL routines::ee key too small
    at exports.createSSLServer (tests/server.js:62:17)
```

Listing 2: tests/test-https.js on macOS.

Newer OpenSSL versions reject 1024-bit keys and SHA-1 signatures by default. We found several such files in the test infrastructure:

- The renewed CA certificate still used the original 1024-bit CA key.
- tests/ssl/test.key is 1024-bit, and tests/ssl/test.crt was signed with SHA-1 and expired in 2021.
- tests/ssl/ca/client.crt expired in November 2021, which broke the mutual TLS tests in tests/test-tunnel.js.

We confirmed the cause by lowering the OpenSSL security level for one run which made all 20 assertions pass. This was only used for diagnosis, since it weakens certificate checking.

1.3 Running the complete test suite

After the CA renewal described in 1.1, we ran the complete test suite on Windows with Node.js 22.19.0. npm run test-ci only reported total 0/1. We later found that this happens because npm runs the script through cmd.exe, which does not expand the wildcard in taper tests/test-*.js. Taper therefore looked for a file literally named test-*.js and ran no tests at all. This probably also explains why test-ci in Report 1 did not identify the failing test file.

We therefore ran the 55 test files individually. 52 files passed (1,434 assertions, no failures), and three failed: tests/test-form-data.js, tests/test-tunnel.js and tests/test-unix.js. Each of the three had a different cause:

- tests/test-tunnel.js crashed with ERR_SSL_CA_MD_TOO_WEAK. Its localhost certificate was signed with SHA-1, and its client certificate expired in November 2021.
- tests/test-form-data.js failed in the "multipart formData + basic auth" case with ECONNRESET, on every run and also on Node.js 24. The test server answers the first request with 401 without reading the multipart body. Our explanation was that modern Node.js closes the connection while the client is still sending the body. This was supported when making the server close the connection explicitly on the 401 response fixed the test (see 1.4).
- tests/test-unix.js failed because it uses a file path as a Unix socket, which does not work on Windows. The original project only ran its CI on Linux.

The browser tests could not be started on Windows (spawn karma ENOENT), and they also depend on PhantomJS, which is no longer maintained. We also noticed that package-lock.json is ignored by .gitignore, so the installed dependencies can differ between group members. A fresh install gave 896 packages and 61 vulnerabilities, compared with 892 and 59 in Report 1.

1.4 Further fixes

Based on these findings, we made the following changes to the test infrastructure:

- Regenerated the CA with a 2048-bit key, and reissued the server, client and localhost certificates with SHA-256 signatures.
- Changed tests/test-unix.js to use a Windows named pipe on Windows, since Unix sockets are not available there. Linux and macOS still use a Unix socket.
- Made the form-data test server close the connection after its 401 response.
- Changed tests/browser/start.js so that the browser tests can start Karma on Windows.

1.5 Current results

On Windows, lint, all 55 test files (1,585 assertions) and the browser tests now pass. On macOS, 50 of 55 test files pass, and test-unix.js still passes with the Unix socket.

The five files that still fail on macOS all use the HTTPS test server in tests/server.js. They crash because tests/ssl/test.key is still a 1024-bit key.

Martin Ek, Andreas Lund, Prodip Kumar Das

October 12, 2026

2 Outlook

The strict HTTPS tests now work again with certificate validation enabled, and the full test suite passes on Windows. The main remaining problem is that the tests still depend on weak keys that newer OpenSSL versions reject.

2.1 Adjustments to the plan

In Progress Report 1, we planned on renewing one expired certificate and then running the full test suite, but the macOS verification showed that the problem was larger. Therefore the scope now covers the whole test certificate set, not only the CA certificate. We also made two test changes that are not about certificates: the Windows named pipe in test-unix.js and the connection handling in test-form-data.js.

We also added a new requirement: a change only counts as verified when the tests pass on both Windows and macOS. For this reason, we decided to postpone the remaining key compatibility and serial number fixes to the final report, so that they can be verified on both systems first.

2.2 Planned maintenance types

Based on the work performed so far, we still expect the project to involve the same three maintenance types as in Progress Report 1. The final classification will be based on the changes that are actually implemented.

- **Adaptive maintenance:** adapt the obsolete test infrastructure to the modern execution environment while preserving strict certificate validation. This includes stricter OpenSSL defaults on newer systems and Windows support for the socket and browser tests.
- **Corrective maintenance:** restore the intended behavior of the strict HTTPS and tunnel tests by replacing the expired certificates in the test infrastructure.
- **Perfective maintenance:** making the certificate generation scripts reliable, so that the certificates can be regenerated in one step, would improve the maintainability of the test infrastructure.

At this point, these classifications are still preliminary. We will only finalize them once the changes and their effect have been verified on both Windows and macOS.

2.3 Next steps

- Replace tests/ssl/test.key with a 2048-bit key. The old key must be kept for tests/test-oauth.js, since its expected signatures depend on it.
- Fix the certificate generation scripts, which currently give the server, client and localhost certificates the same serial number.
- Decide whether the psl dependency override in package.json is needed.
- Check the certificates used by the browser tests, which are 1024-bit and expired in January 2025.

- Rerun the full test suite on both Windows and macOS, and document the final results and maintenance types.

3 Use of generative AI tools

As permitted in the course provided that its use is documented, all group members used generative AI tools during this phase including Claude Code and GitHub Copilot. The tools were used to help run and analyze tests, investigate failures, propose changes, and draft documentation in MAINTENANCE.md.

- Martin: verifying the certificate fix on macOS, investigating the remaining HTTPS failures, and drafting the documentation for said findings.
- Andreas: investigating and documenting the three remaining failures, setting up a matching Node.js 22 environment.
- Prodip: using GitHub Copilot while working on the certificate and test infrastructure changes, and documentation.

The main benefit of using this was speed. Running all 55 test files and inspecting every certificate took minutes, and this way dependencies were also found that we might have missed, such as tests/test-oauth.js relying on the old test key.

The main challenge was that it sometimes stated incorrect conclusions with confidence, which for example happened when the renewed CA was first described as correct, before its 1024-bit key was found.

Seeing as AI tools can present incorrect information as fact, key results were checked and reproduced manually, and changes made were also reviewed through pull requests before being accepted.